New Horizons
Centro de Capacitación

Práctica: Formulario Reactivo Base de Solicitud y Validaciones

Paso a paso — Configurar ReactiveFormsModule, FormBuilder, validaciones sincrónicas y feedback de error en Material

Curso	Técnico de Desarrollo Full Stack con C# y Angular
Sesión	Sesión 10 — Formularios Reactivos y Validaciones Avanzadas
Docente	Edwin Barrientos
Duración estimada	60–90 minutos
Nivel	Básico (guía principal de la sesión)
Entorno	.NET 8 SDK · Visual Studio 2022 Community o VS Code · PostgreSQL o SQL Server Express

1. Objetivos

- Configurar ReactiveFormsModule en componentes standalone de Angular 18.
- Construir un FormGroup mediante la inyección de FormBuilder con campos motivo y centro de costo.
- Aplicar validadores nativos (Validators.required, minLength, maxLength) como espejo de FluentValidation.
- Crear un validador personalizado sincrónico para controlar longitud y caracteres permitidos.
- Visualizar errores de validación en tiempo real con mat-error y estados touched/dirty.

2. Desarrollo paso a paso

Paso 1 — Configurar el Modelo de DTO para Formularios

Definimos las interfaces de entrada en src/app/models/solicitud.model.ts que coinciden con CrearSolicitudDto de C#:

src/app/models/solicitud.model.ts

```
export interface CrearSolicitudDetalleDto {
  concepto: string;
  cantidad: number;
  precioUnitario: number;
}

export interface CrearSolicitudDto {
  motivo: string;
  centroCostoId: number;
  solicitanteId: number;
  detalles: CrearSolicitudDetalleDto[];
}
```

Paso 2 — Crear Validador Personalizado sincrónico

Implementamos una función validadora en src/app/validators/custom-validators.ts para verificar que el motivo contenga al menos 3 palabras descriptivas:

src/app/validators/custom-validators.ts

```
import { AbstractControl, ValidationErrors, ValidatorFn } from '@angular/forms';

export function motivoMinimoPalabrasValidator(minPalabras: number): ValidatorFn {
  return (control: AbstractControl): ValidationErrors | null => {
    if (!control.value) return null;
    const palabras = control.value.trim().split(/\s+/).filter((p: string) => p.length > 0);
    return palabras.length >= minPalabras ? null : {
      motivoInsuficiente: { palabrasActuales: palabras.length, requeridas: minPalabras }
    };
  };
}
```

Paso 3 — Implementar SolicitudFormComponent con FormBuilder

Creamos el componente en src/app/pages/solicitud-form/ separando la lógica (.ts), plantilla (.html) y estilos (.css):

src/app/pages/solicitud-form/solicitud-form.component.ts

```
import { Component, OnInit, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormBuilder, FormGroup, ReactiveFormsModule, Validators } from '@angular/forms';
import { Router, RouterLink } from '@angular/router';
import { MatFormFieldModule } from '@angular/material/form-field';
import { MatInputModule } from '@angular/material/input';
import { MatSelectModule } from '@angular/material/select';
import { MatButtonModule } from '@angular/material/button';
import { MatCardModule } from '@angular/material/card';
import { MatIconModule } from '@angular/material/icon';
import { SolicitudService } from '../../services/solicitud.service';
import { motivoMinimoPalabrasValidator } from '../../validators/custom-validators';

@Component({
  selector: 'app-solicitud-form',
  standalone: true,
  imports: [
    CommonModule, ReactiveFormsModule, RouterLink,
    MatFormFieldModule, MatInputModule, MatSelectModule, MatButtonModule, MatCardModule,
MatIconModule
  ],
  templateUrl: './solicitud-form.component.html',
  styleUrl: './solicitud-form.component.css'
})
export class SolicitudFormComponent implements OnInit {
  private fb = inject(FormBuilder);
  private router = inject(Router);
  private service = inject(SolicitudService);

  formulario!: FormGroup;

  centrosCosto = [
    { id: 1, codigo: 'CC-001', nombre: 'Tecnologías de la Información' },
    { id: 2, codigo: 'CC-002', nombre: 'Operaciones y Logística' },
    { id: 3, codigo: 'CC-003', nombre: 'Ventas y Marketing' },
    { id: 4, codigo: 'CC-004', nombre: 'Administración y Finanzas' }
  ];

  ngOnInit(): void {
    this.formulario = this.fb.group({
      motivo: ['', [Validators.required, Validators.minLength(10), Validators.maxLength(250),
motivoMinimoPalabrasValidator(3)]],
      centroCostoId: [null, [Validators.required]],
      solicitanteId: [1]
    });
  }

  guardar(): void {
    if (this.formulario.invalid) return;
    const dto = {
      ...this.formulario.value,
      detalles: []
    };
    this.service.crear(dto).subscribe(() => {
      this.router.navigate(['/solicitudes']);
    });
  }
}
```

src/app/pages/solicitud-form/solicitud-form.component.html

```
<div class="form-container">
  <div class="form-header">
    <a mat-button routerLink="/solicitudes">&larr; Volver a la Bandeja</a>
    <h2>Nueva Solicitud de Dinero / Compra</h2>
  </div>

  <mat-card class="form-card mat-elevation-z3">
    <form [formGroup]="formulario" (ngSubmit)="guardar()">
      <div class="field-row">
        <mat-form-field appearance="outline" class="full-width">
          <mat-label>Motivo o Justificación del Gasto</mat-label>
          <textarea matInput formControlName="motivo" rows="3" placeholder="Ej: Compra de
licencias de desarrollo para el equipo"></textarea>
          <mat-error *ngIf="formulario.get('motivo')?.hasError('required')">El motivo es
obligatorio.</mat-error>
          <mat-error *ngIf="formulario.get('motivo')?.hasError('minlength')">Debe tener al menos
10 caracteres.</mat-error>
          <mat-error *ngIf="formulario.get('motivo')?.hasError('motivoInsuficiente')">
            Ingrese una justificación de al menos 3 palabras explicativas.
          </mat-error>
        </mat-form-field>
      </div>

      <div class="field-row-split">
        <mat-form-field appearance="outline" class="half-width">
          <mat-label>Centro de Costo</mat-label>
          <mat-select formControlName="centroCostoId">
            <mat-option *ngFor="let cc of centrosCosto" [value]="cc.id">
              {{ cc.codigo }} – {{ cc.nombre }}
            </mat-option>
          </mat-select>
          <mat-error *ngIf="formulario.get('centroCostoId')?.hasError('required')">
            Seleccione un centro de costo.
          </mat-error>
        </mat-form-field>

        <mat-form-field appearance="outline" class="half-width">
          <mat-label>Solicitante</mat-label>
          <input matInput value="Juan Pérez (TI)" readonly>
        </mat-form-field>
      </div>

      <div class="actions-bar">
        <button mat-button type="button" routerLink="/solicitudes">Cancelar</button>
        <button mat-raised-button color="primary" type="submit" [disabled]="formulario.invalid">
          <mat-icon>save</mat-icon> Guardar Solicitud
        </button>
      </div>
    </form>
  </mat-card>
</div>
```

src/app/pages/solicitud-form/solicitud-form.component.css

```
.form-container { max-width: 800px; margin: 0 auto; padding: 12px; }
.form-header { margin-bottom: 16px; }
.form-card { padding: 24px; border-radius: 12px; background: white; }
.full-width { width: 100%; margin-bottom: 12px; }
.field-row-split { display: flex; gap: 16px; margin-bottom: 16px; }
.half-width { flex: 1; }
.actions-bar { display: flex; justify-content: flex-end; gap: 12px; margin-top: 24px; }
```

3. Retos adicionales

1. Implementa un validador asincrónico que verifique si el código o motivo ya fue registrado recientemente.
2. Añade un contador visual de caracteres restantes en el campo motivo (ej. 45/250).
3. Agrega una alerta visual antes de abandonar la pantalla si el formulario tiene cambios sin guardar (dirty).
4. Añade un método para restablecer los valores por defecto del formulario con confirmación.

4. Entregables

- Formulario reactivo creado con FormBuilder e inyección limpia con inject().
- Validadores estándar y validador personalizado motivoMinimoPalabrasValidator activos.
- Visualización correcta de mat-error solo cuando el campo ha sido tocado o modificado.
- Botón de guardar deshabilitado reactivamente mientras el formulario sea inválido.